

HelixTrack

HelixTrack

JIRA का मुक्त-विश्व विकल्प

सारांश

HelixTrack एक व्यापक, आधुनिक और ओपन-सोर्स विकल्प है JIRA (और इसके Documents एक्सटेंशन के माध्यम से Confluence) का — एक बहु-प्लेटफॉर्म प्रोजेक्ट-प्रबंधन और समस्या-ट्रैकिंग प्रणाली, जो Go माइक्रोसर्विसेज बैकएंड पर आधारित है और वेब, डेस्कटॉप तथा मोबाइल के लिए नेटिव क्लाइंट्स के साथ आती है।

संक्षिप्त विवरण

ओपन-सोर्स JIRA/Confluence विकल्प। एक Go माइक्रोसर्विसेज बैकएंड (“HelixTrack कोर”) एकीकृत REST API इंटरफ़ेस प्रदान करता है प्रोजेक्ट और समस्या ट्रैकिंग के लिए, साथ ही Confluence-शैली का दस्तावेज कार्यक्षेत्र, जिसे नेटिव वेब, डेस्कटॉप, एंड्रॉयड और आईओएस क्लाइंट्स को HTTP/3 QUIC के माध्यम से परोसा जाता है।

विस्तृत विवरण

HelixTrack एक ओपन-सोर्स प्रोजेक्ट-प्रबंधन और समस्या-ट्रैकिंग प्लेटफॉर्म है, जिसे JIRA और Confluence का मुक्त-विश्व विकल्प के रूप में प्रस्तुत किया गया है — उन दो उत्पादों का पूर्ण प्रतिस्थापन, जिन पर अधिकांश इंजीनियरिंग संगठन निर्भर हैं, लेकिन जिन्हें अब आपकी अपनी सॉफ्टवेयर के रूप में पुनर्निर्मित किया गया है, जिसे आप कहीं भी चला सकते हैं। इसका केंद्र है **HelixTrack कोर**, एक REST API माइक्रोसर्विस जो Go में Gin फ्रेमवर्क के साथ लिखी गई है। यह पूर्ण समस्या ट्रैकिंग, एजाइल/स्क्रम बोर्ड, टीम प्रबंधन और एक पदानुक्रमित अनुमति इंजन प्रदान करता है, जिसकी कार्यप्रणाली को स्थानीय इन-प्रोसेस इंजन और HTTP-समर्थित सेवा के बीच बदला जा सकता है — ताकि एक ही प्राधिकरण मॉडल एकल लैपटॉप से लेकर वितरित क्लस्टर तक बिना एप्लिकेशन कोड में बदलाव किए स्केल कर सके। दर्जनों रूट्स पर REST इंटरफ़ेस फैलाने के बजाय, कोर सब कुछ एक ही एक्शन-रूटेड /do एंडपॉइंट के माध्यम से संचालित करता है, जिसमें एक समान अनुरोध/प्रतिसाद संरचना (action/jwt/object/data इनपुट, errorCode/errorMessage/data आउटपुट) होती है: हर क्लाइंट एक ही छोटे अनुबंध का पालन करता है, और किसी नई क्षमता को जोड़ने का मतलब एक नया एक्शन जोड़ना है, न कि एक नया URL जिसे दस्तावेजीकृत, सुरक्षित और संस्करणबद्ध करना पड़े। कोर डिकपल्ड

ऑथेंटिकेशन, अनुमति और स्थानीयकरण सेवाओं के साथ एकीकृत होता है, जो HTTP/3 QUIC पर संचार करती हैं और अलग-अलग मशीनों या क्लस्टरों पर चल सकती हैं, या परीक्षण विन्यासों में पूरी तरह बंद की जा सकती हैं। डेटा SQLite में शून्य-सेटअप विकास के लिए और PostgreSQL में उत्पादन के लिए संग्रहीत होता है, और आराम से एन्क्रिप्टेड रहता है SQLCipher (AES-256) के साथ, ताकि संवेदनशील प्रोजेक्ट डेटा डिस्क पर डिफ़ॉल्ट रूप से सुरक्षित रहे, न कि बाद में सोच-विचार करके। **Documents V2** एक्सटेंशन ट्रैकर को एक पूर्ण ज्ञान प्लेटफ़ॉर्म में बदल देता है: एक Confluence-शैली का कार्यक्षेत्र जिसमें स्पेस, पृष्ठ, संस्करण नियंत्रण, टेम्प्लेट, रीयल-टाइम WebSocket सहयोग और विश्लेषण शामिल हैं — विकी और समस्या ट्रैकर अंततः एक ही बैकएंड के पीछे आ जाते हैं, बजाय दो अलग-अलग उत्पादों को जोड़ने के। कोर के चारों ओर कई क्लाइंट एप्लिकेशन मौजूद हैं: एक Angular वेब क्लाइंट, एक Tauri + Angular डेस्कटॉप क्लाइंट, नेटिव एंड्रॉयड (Kotlin) और आईओएस (Swift) ऐप्स, साथ ही HarmonyOS और Aurora OS क्लाइंट्स और एक स्क्रीनसेवर — सभी एक ही बैकएंड से संवाद करते हैं और UDP प्रसारण के माध्यम से स्थानीय नेटवर्क पर स्वचालित रूप से इसे खोज लेते हैं, ताकि एक नया क्लाइंट बिना मैन्युअल कॉन्फिगरेशन के अपना सर्वर ढूँढ़ सके। क्लाइंट एप्लिकेशन अलग-अलग निजी रिपॉजिटरी में रखे जाते हैं और यहाँ केवल उत्पाद स्तर पर प्रस्तुत किए गए हैं।

सामग्री

हमने इसे क्यों बनाया

टीमों को JIRA + कॉन्फ्लुएंस के ढेर के लिए एक वास्तविक रूप से खुला, स्व-होस्ट करने योग्य विकल्प देना — “स्वतंत्र दुनिया के लिए” — बिना किसी विक्रेता लॉक-इन के, जिसमें उद्यम-स्तरीय ट्रैकिंग, दस्तावेज़ और सहयोग को एक ही ओपन-सोर्स लाइसेंस के तहत समाहित किया गया हो।

यह क्यों गेम-चेंजर है

यह दो भारी-भरकम व्यावसायिक उत्पादों — इश्यू ट्रैकिंग और विकी/दस्तावेज़ ढेर — को एक खुले, उच्च-प्रदर्शन वाले, स्व-होस्ट करने योग्य प्लेटफ़ॉर्म में समेट देता है, और इसके साथ वह चीज़ जो मौजूदा प्रतिस्पर्धियों ने कभी नहीं दी: सच्चे मल्टी-प्लेटफ़ॉर्म नेटिव क्लाइंट (वेब, डेस्कटॉप, एंड्रॉयड, आईओएस, साथ ही हार्मनीओएस और ऑरोरा ओएस) — सभी एक ही बैकएंड कॉन्ट्रैक्ट द्वारा संचालित। असली बदलाव है बिना किसी समझौते के स्वामित्व। HTTP/3-हर जगह, पूरी तरह से डिफ़ॉल्ट माइक्रोसर्विस डिज़ाइन और आराम की स्थिति में SQLCipher AES-256 एन्क्रिप्शन, जो आमतौर पर प्रोप्राइटी सास के लिए आरक्षित प्रदर्शन और सुरक्षा को एक ऐसी प्रणाली में लाता है जिसे आप खुद होस्ट करते हैं — बिना सीट लाइसेंस के, बिना विक्रेता लॉक-इन के, बिना आपके इंफ्रास्ट्रक्चर से डेटा बाहर जाने के। टीमों को वह JIRA-प्लस-कॉन्फ्लुएंस अनुभव मिलता है जिसे वे पहले से जानते हैं, अपने खुद के हार्डवेयर पर, एक ही ओपन-सोर्स लाइसेंस के तहत।

क्या है नवाचारी

- एकीकृत क्रिया-आधारित /do API — एक एंडपॉइंट, एक लिफाफा, क्रिया द्वारा रूट किया गया। नई क्षमताएँ नए यूआरएल के रूप में नहीं, बल्कि नई क्रियाओं के रूप में आती हैं, जिससे हमले की सतह, क्लाइंट कोड और दस्तावेज़ीकरण का बोझ एक ही कॉन्ट्रैक्ट में सिमट जाता है जिसे हर प्लेटफ़ॉर्म साझा करता है।

- HTTP/3 QUIC को डिफ़ॉल्ट इंटर-सर्विस ट्रांसपोर्ट के रूप में—आधुनिक कम-विलंबता, कनेक्शन-लचीला नेटवर्किंग सेवाओं के बीच पहले दिन से ही, बाद में जोड़ा नहीं गया।
- एक अनुमतियों वाला इंजन जो स्थानीय इन-प्रोसेस कार्यान्वयन और एचटीटीपी-समर्थित सेवा के बीच बदला जा सकता है, साथ ही वैकल्पिक, स्वतंत्र रूप से तैनात करने योग्य ऑथ, अनुमतियाँ और स्थानीयकरण सेवाएँ—वही प्राधिकरण मॉडल चाहे आप एक प्रक्रिया चलाएँ या एक क्लस्टर।
- --space-root फ़्लैग के माध्यम से मल्टी-स्पेस डेटा आइसोलेशन: हर प्रोजेक्ट को अपना अलग डेटाबेस और एसेट स्टोर मिलता है, ताकि किराएदार और प्रोजेक्ट स्टोरेज सीमा पर ही अलग हो जाएँ, न कि क्वेरी फ़िल्टर के ज़रिए।
- आराम की स्थिति में SQLCipher AES-256 एन्क्रिप्शन—संवेदनशील प्रोजेक्ट डेटा डिस्क पर डिफ़ॉल्ट रूप से पारदर्शी तरीके से सुरक्षित रहता है।
- स्थानीय नेटवर्क पर UDP प्रसारण के माध्यम से स्वचालित क्लाइंट-टू-सर्वर खोज—क्लाइंट बिना किसी मैनुअल कॉन्फ़िगरेशन के कोर को ढूँढ़ लेता है।
- दस्तावेज़ V2, एक वास्तविक “कॉन्फ्लुएंस विकल्प”, जिसमें आशावादी-लॉकिंग समानांतर संपादन, टकराव का पता लगाना और पूरा बदलाव इतिहास शामिल है—वास्तविक सहयोगी दस्तावेज़ जो ट्रैकर के समान बैकएंड के पीछे रहते हैं।

सबसे बड़ी तकनीकी चुनौतियाँ और हमने उन्हें कैसे हल किया

- छह क्लाइंट प्लेटफ़ॉर्म, एक बैकएंड, शून्य कॉन्टैक्ट ड्रिफ्ट। वेब/Angular, डेस्कटॉप/Tauri, एंड्रॉयड/Kotlin, आईओएस/Swift, हार्मनीओएस और ओरोरा क्लाइंट्स को बनाए रखना आमतौर पर छह अलग-अलग API एकीकरणों का मतलब होता है जो सिंक से बाहर हो जाते हैं। हमने इस जोखिम को एक ही क्रिया-रूटेड /do API और उसके निश्चित लिफाफे को एकमात्र कॉन्टैक्ट बनाकर समाप्त कर दिया—हर क्लाइंट इसे एक समान तरीके से लक्षित करता है—और ऊपर से UDP प्रसारण सेवा खोज को जोड़ दिया ताकि क्लाइंट बिना मैनुअल कॉन्फ़िगर किए नेटवर्क पर कोर का पता लगा सकें।
- सेवाओं को डिक्पल किए बिना विलंबता का भुगतान न करना। ऑथ, अनुमतियाँ और स्थानीयकरण को स्वतंत्र रूप से तैनात करने योग्य सेवाओं में विभाजित करने से आमतौर पर हर कॉल पर एक नेटवर्क हॉप जुड़ जाता है। हमने सभी इंटर-सर्विस कॉल्स में HTTP/3 QUIC को अपनाया ताकि वे हॉप्स तेज़ और कनेक्शन-लचीले रहें, और हर सेवा को स्वतंत्र रूप से चलाने योग्य बनाया—यहाँ तक कि परीक्षण कॉन्फ़िगरेशन में पूरी तरह अक्षम भी—ताकि डिक्पलिंग एक तैनाती विकल्प हो, स्थिर लागत नहीं।
- कॉन्फ्लुएंस-स्तरीय सहयोग बिना खोए हुए लेखन के गड़बड़ के। रीयल-टाइम मल्टी-ऑथर संपादन टकराव वाले लेखन को आमंत्रित करता है। दस्तावेज़ V2 इसे स्पेस/पेज/वर्जनिंग के साथ आशावादी लॉकिंग, स्पष्ट टकराव का पता लगाने, पूरा बदलाव इतिहास और रीयल-टाइम WebSocket सिंक के ज़रिए हल करता है—सहयोग जो संगत बना रहता है बजाय चुपचाप संपादनों को मिटा देने के।
- आराम की स्थिति में एन्क्रिप्शन बिना थ्रूपुट को मार दिए। SQLCipher AES-256 डिस्क पर डेटा की सुरक्षा करता है लेकिन प्रति-क्वेरी ओवरहेड जोड़ता है; हमने इसे मल्टी-लेयर कैशिंग (स्थानीयकरण सेवा में Redis के सामने इन-मेमोरी LRU) से संतुलित किया ताकि मल्टी-लैंग्वेज लुकअप जैसी हॉट पाथ तेज़ रहें जबकि डेटा एन्क्रिप्टेड बना रहे।

तकनीकी स्टैक

- **Go + Gin** — उच्च थ्रूपुट और कम विलंबता वाले HTTP सेवाओं के लिए चुना गया, जिसमें सिंगल-बाइनरी डिप्लॉयमेंट की सुविधा है; यह कोर के REST API को समाहित करता है, इसके JWT/CORS मिडलवेयर को, और संपूर्ण सिस्टम के सामने आने वाले एक्शन-रूटेड /do राउटर को।
- **HTTP/3 QUIC** — कोर और उसके ऑथ/परमिशन/लोकलाइजेशन सेवाओं के बीच ट्रांसपोर्ट के रूप में चुना गया, क्योंकि QUIC का मल्टीप्लेक्सड, कनेक्शन-माइग्रेटिंग डिज़ाइन टेल लेटेंसी को कम करता है और TCP की तुलना में अस्थिर कनेक्शनों पर भी काम करता है जहाँ TCP रुक जाता है।
- **PostgreSQL (प्रोडक्शन) / SQLite (डेवलपमेंट)** — एक रिलेशनल मॉडल जो दोनों इंजनों पर बड़े ट्रैकिंग-प्लस-डॉक्यूमेंट्स स्कीमा को सपोर्ट करता है: SQLite स्थानीय विकास को शून्य-सेटअप और फ़ाइल-आधारित रखता है, जबकि प्रोडक्शन स्केल के लिए पोस्टग्रेस एक समर्पित production कंपोज़ प्रोफ़ाइल के माध्यम से कार्यभार संभालता है।
- **SQLCipher (AES-256)** — पारदर्शी, डेटाबेस-स्तरीय एन्क्रिप्शन के लिए चुना गया, ताकि संवेदनशील प्रोजेक्ट डेटा की सुरक्षा के लिए एप्लिकेशन-लेयर क्रिप्टो की आवश्यकता न पड़े और क्वेरी लिखने के तरीके में कोई बदलाव न करना पड़े।
- **Redis** — लोकलाइजेशन सेवा के पीछे इन-मेमोरी LRU के साथ साझा कैश टियर के रूप में चुना गया, जो दो-परत कैश प्रदान करता है और मल्टी-लैंग्वेज लुकअप को तेज़ रखता है, भले ही नीचे एन्क्रिप्शन ओवरहेड हो।
- **Uber Zap + Lumberjack** — संरचित, अलोकेशन-लाइट लॉगिंग के लिए चुना गया, जिसमें बिल्ट-इन रोटेशन है, ताकि कोर प्रोडक्शन में ऑब्जर्वेबल रहे और लॉग का आकार असीमित न बढ़े।
- **golang-jwt / JWT** — स्टेटलेस ऑथ मैकेनिज्म के रूप में चुना गया; हस्ताक्षरित टोकन हर /do एनवेलप के jwt फ़ील्ड में होता है, जिससे सभी क्लाइंट्स में प्रमाणीकरण एक समान रहता है।
- **Angular 19 (+ Material, RxJS)** — रिएक्टिव, कंपोनेंट-ड्रिवेन ब्राउज़र क्लाइंट के लिए चुना गया, जिसमें आउट-ऑफ़-द-बॉक्स परिपक्व मटेरियल डिज़ाइन सिस्टम है।
- **Tauri 2.0 + Rust + Angular** — एक नेटिव डेस्कटॉप शेल के लिए चुना गया, जिसमें Angular UI को Rust-बैकड वेबव्यू के अंदर पुनः उपयोग करके एक छोटा फुटप्रिंट होता है, बजाय पूरे ब्राउज़र रनटाइम को बंडल करने के।
- **Kotlin (एंड्रॉयड) / Swift + SwiftUI (iOS)** — मोबाइल उपयोगकर्ताओं के लिए चुना गया, ताकि उन्हें रेण्ड वेब व्यू के बजाय वास्तविक नेटिव, प्लेटफ़ॉर्म-आइडियोमैटिक क्लाइंट मिलें।
- **Docker / Docker Compose (Podman-संगत)** — पुनरुत्पादन योग्य, कंटेनरीकृत डिप्लॉयमेंट के लिए चुना गया, जिसमें /health चेक जुड़े हुए हैं, और Podman संगतता के कारण किसी डेमन या वेंडर की अनिवार्यता नहीं है।
- **Testify (Go); Cypress/Playwright/Karma+Jasmine (क्लाइंट्स)** — स्तरित स्वचालित परीक्षण के लिए चुना गया, जो बैकएंड कॉन्ट्रैक्ट और क्लाइंट UI को स्वतंत्र रूप से कवर करता है, जो सिंगल-बैकएंड/मल्टी-क्लाइंट आर्किटेक्चर के अनुरूप है।

स्थिति और ईमानदारी संबंधी टिप्पणियाँ

- **स्थिति:** बीटा। HelixTrack कोर एक कार्यशील REST API माइक्रोसर्विस है; डॉक्यूमेंट्स V2 एक्सटेंशन को लगभग 95% पूर्ण दर्ज किया गया है, जिसमें एक ज्ञात

डेटाबेस फ़्रील्ड-मैपिंग समस्या है, इसलिए इसे पूरी तरह से शिफ्ट के रूप में प्रस्तुत नहीं किया गया है।

- **लाइसेंस: निर्धारित नहीं।** CLAUDE.md में MIT लिखा है, लेकिन core/LICENSE फ़ाइल में अपाचे 2.0 दर्ज है — इस विसंगति का समाधान लाइसेंस को अंतिम रूप देने से पहले किया जाना आवश्यक है।
- प्रोजेक्ट के README में उद्धृत प्रदर्शन आँकड़े (जैसे 50,000+ अनुरोध/सेकंड, मिलीसेकंड से कम क्वेरी समय) डिज़ाइन/मार्केटिंग लक्ष्य हैं, स्वतंत्र रूप से प्रकाशित बेंचमार्क नहीं, इसलिए इन्हें उपरोक्त दावों में शामिल नहीं किया गया है।
- क्लाउंट एप्लिकेशन (वेब, डेस्कटॉप, एंड्रॉयड, iOS, ऑरोरा, हार्मनीOS) निजी रिपॉजिटरी में हैं और केवल उत्पाद स्तर पर वर्णित हैं।

सामग्री

प्राथमिकता स्तर: Helix-प्राथमिक और Helix-ट्रैक उत्पाद शृंखला का प्रमुख — किसी भी Server Factory परियोजना से पहले स्थान पर।